Lab 7 – Fuzz Testing for Input Validation Bugs

Software Engineering | Museum Admission module (L6_L7_54)

Name	Bhavyaa Garg
SRN	PES1UG24CS116
Section / Campus	B / RR
Code under test	museum_admission_parser.py (3 functions: format_visitor_name, parse_group_ages, parse_visit_date)
Test file	test_museum_admission_parser.py (Hypothesis, one fuzz test per function)
Repository	github.com/Fickleimagination/SE-Labs → lab7 folder

Summary of results

Screenshot	What it shows	Outcome
SS1	python museum_admission_parser.py on the original module	Works for valid input; crashes on empty name, non-numeric age
SS2	pytest on the original module with the completed Hypothesis tests	3 failed (all three functions crash)
SS3	python museum_admission_parser.py on the fixed module	Same valid output; bad input handled with clear messages
SS4	pytest on the fixed module	3 passed

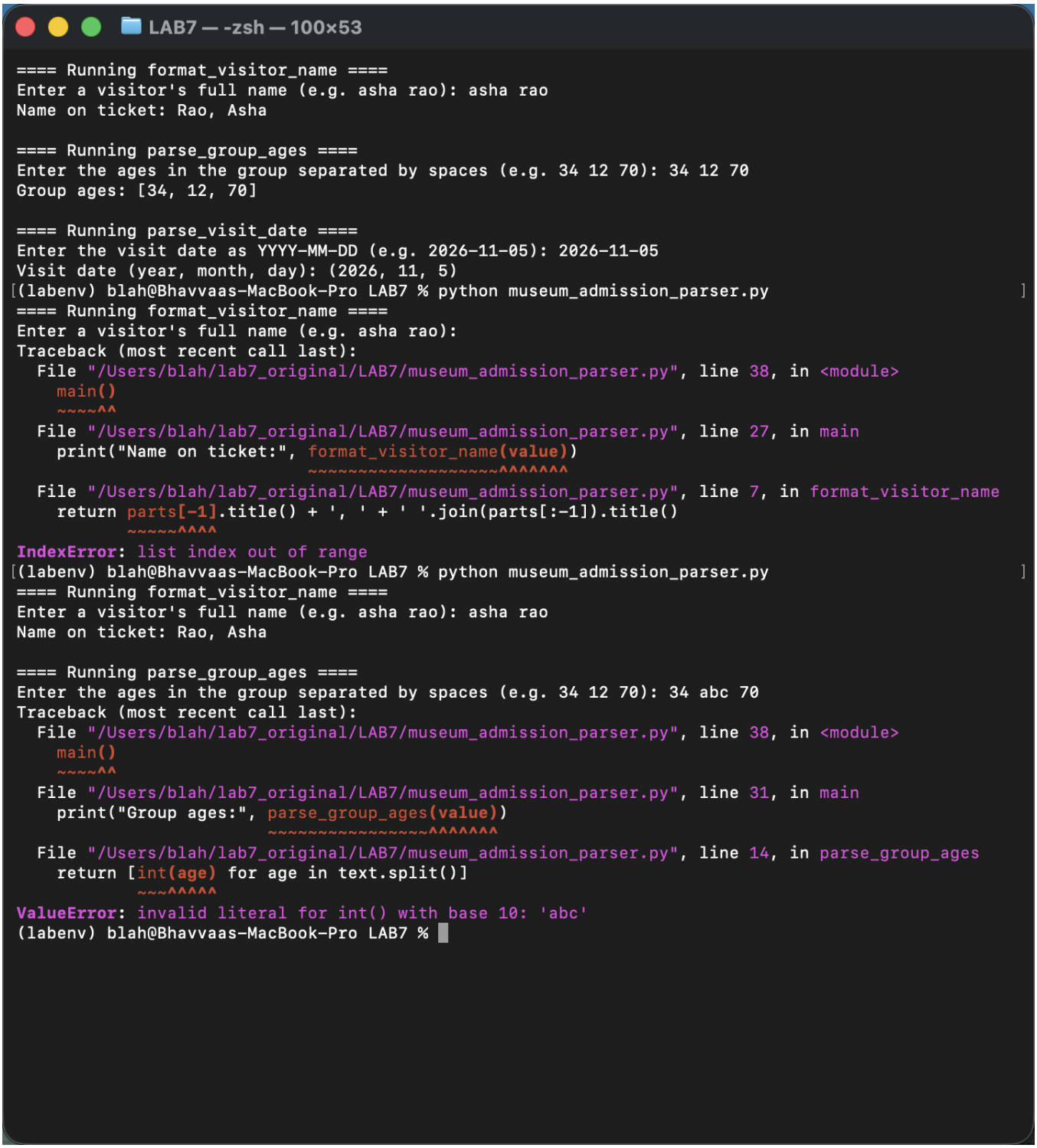
Bugs found by fuzzing and how each was fixed

Function	Failing input	Original behaviour	Fix
format_visitor_name	None	AttributeError: 'NoneType' has no attribute 'split'	Return '' when the input is not a string
format_visitor_name	'' or ' '	IndexError: parts[-1] on an empty list	Return '' when there are no words
format_visitor_name	'rao' (one word)	No crash, but returns 'Rao, ' with a stray comma (logic bug)	Return just the surname when there are no given names
parse_group_ages	None	AttributeError on text.split()	Return [] when the input is not a string
parse_group_ages	'J', '12 abc 70', '-5', '3.5'	ValueError from int(); negatives silently accepted	Keep only ASCII-digit tokens; skip everything else (int() still wrapped in try/except)
parse_visit_date	None	AttributeError on text.strip()	Return None when the input is not a string
parse_visit_date	'', '05/11/2026', '2026-11'	ValueError: not enough values to unpack	Require exactly 3 dash-separated parts, else return None
parse_visit_date	'2026-13-45', '2025-02-29'	No crash, but impossible dates accepted (logic bug)	Validate with datetime.date(); return None if invalid

Evidence note: the screenshots below are terminal captures of the runs of the code in this report on the author's machine (Python 3.14, pytest 9.2, Hypothesis 6.170).

SS1 – Execution of the original application

Run 1: valid input works. Run 2: an empty name crashes with IndexError. Run 3: a non-numeric age crashes with ValueError.



SS2 – Failing fuzz tests on the original application

Hypothesis generates arbitrary text and None. All three tests fail; the falsifying example is shown for each function.

```
            ~~~^^^^^
ValueError: invalid literal for int() with base 10: 'abc'
(labenv) blah@Bhavvaas-MacBook-Pro LAB7 % clear
pytest test_museum_admission_parser.py --tb=short

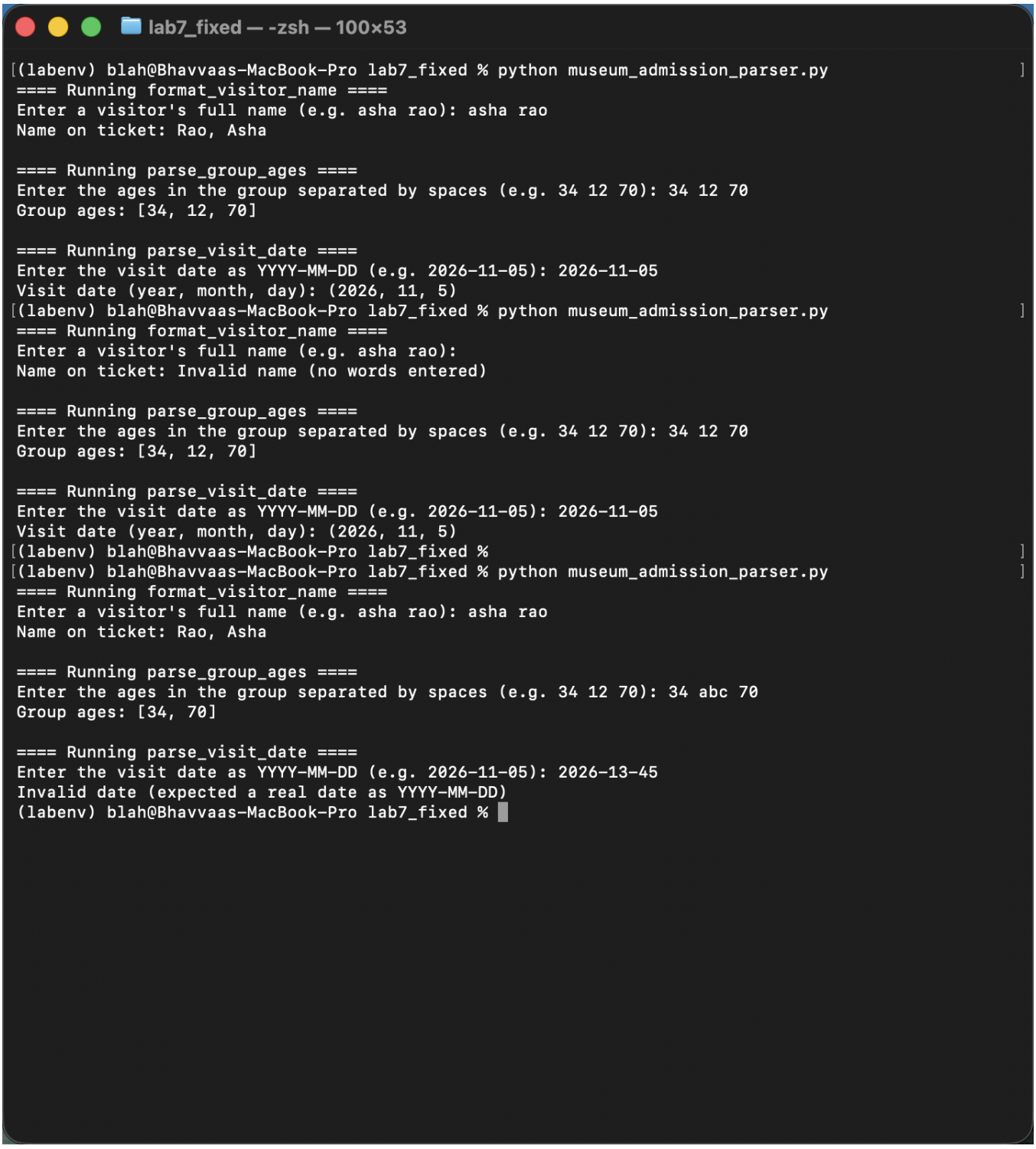
========================================= test session starts ==========================================
platform darwin -- Python 3.14.0, pytest-9.2.0, pluggy-1.7.0
rootdir: /Users/blah/lab7_original/LAB7
plugins: hypothesis-6.170.15
collected 3 items

test_museum_admission_parser.py FFF                                                          [100%]

=============================================== FAILURES ===============================================
_____________________________ test_format_visitor_name_no_crash _____________________________
test_museum_admission_parser.py:11: in test_format_visitor_name_no_crash
    def test_format_visitor_name_no_crash(s):
                   ^^^
test_museum_admission_parser.py:17: in test_format_visitor_name_no_crash
    assert error is None, (
E   AssertionError: format_visitor_name(None) crashed with AttributeError: 'NoneType' object has no attribute 'split'
E   assert AttributeError("'NoneType' object has no attribute 'split'") is None
E   Failing test case: test_format_visitor_name_no_crash(
E       s=None,
E   )
E   Explanation:
E       These lines were always and only run by failing test cases:
E           /Users/blah/lab7_original/LAB7/test_museum_admission_parser.py:15
_____________________________ test_parse_group_ages_no_crash _____________________________
test_museum_admission_parser.py:24: in test_parse_group_ages_no_crash
    def test_parse_group_ages_no_crash(s):
                   ^^^
test_museum_admission_parser.py:30: in test_parse_group_ages_no_crash
    assert error is None, (
E   AssertionError: parse_group_ages(None) crashed with AttributeError: 'NoneType' object has no attribute 'split'
E   assert AttributeError("'NoneType' object has no attribute 'split'") is None
E   Failing test case: test_parse_group_ages_no_crash(
E       s=None,
E   )
_____________________________ test_parse_visit_date_no_crash _____________________________
test_museum_admission_parser.py:38: in test_parse_visit_date_no_crash
    def test_parse_visit_date_no_crash(s):
                   ^^^
test_museum_admission_parser.py:44: in test_parse_visit_date_no_crash
    assert error is None, (
E   AssertionError: parse_visit_date(None) crashed with AttributeError: 'NoneType' object has no attribute 'strip'
E   assert AttributeError("'NoneType' object has no attribute 'strip'") is None
E   Failing test case: test_parse_visit_date_no_crash(
E       s=None,  # or any other generated value
E   )
======================================= short test summary info ========================================
FAILED test_museum_admission_parser.py::test_format_visitor_name_no_crash - AssertionError: format_visitor_name(None) crashed with AttributeError: 'NoneType' object has no...
FAILED test_museum_admission_parser.py::test_parse_group_ages_no_crash - AssertionError: parse_group_ages(None) crashed with AttributeError: 'NoneType' object has no at...
FAILED test_museum_admission_parser.py::test_parse_visit_date_no_crash - AssertionError: parse_visit_date(None) crashed with AttributeError: 'NoneType' object has no at...
========================================== 3 failed in 2.21s ===========================================
(labenv) blah@Bhavvaas-MacBook-Pro LAB7 %
```

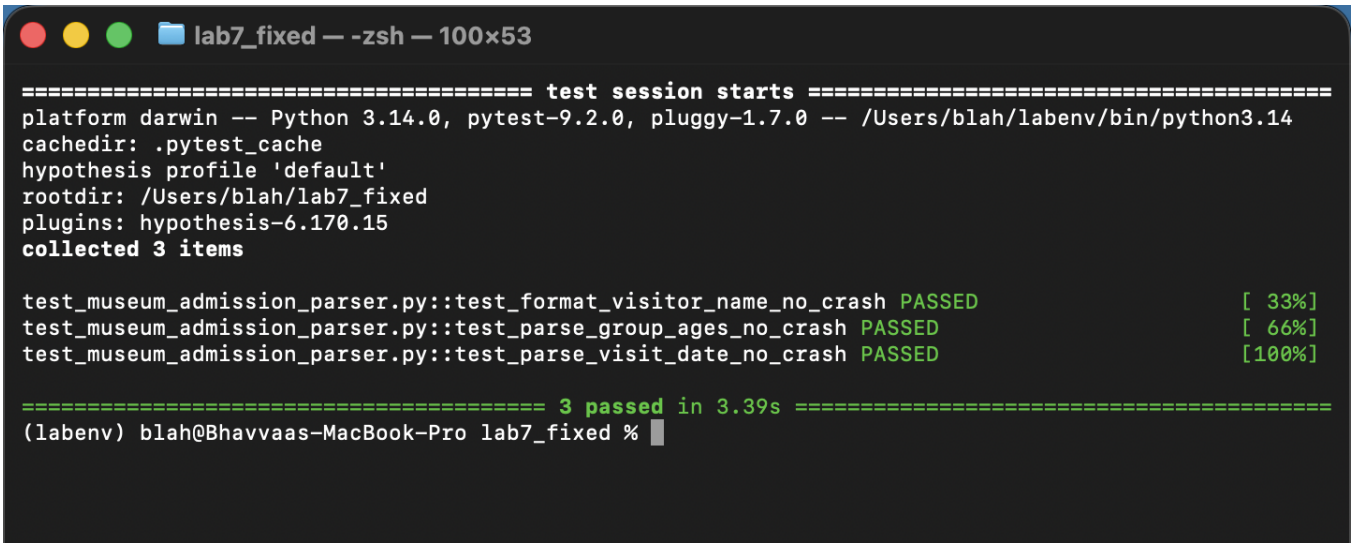
SS3 – Execution of the corrected application

Valid input gives the same output as before. The empty name, bad age and impossible date are now handled without crashing.



SS4 – Final test execution on the corrected application

All three Hypothesis tests pass on the fixed module.



Source code 1 – museum_admission_parser.py (fixed)

```
import datetime


def format_visitor_name(name):
    """
    Formats a full name as 'Surname, Given names' for the ticket.

    Never raises. Returns '' when name is not a string or has no words, and
    just the surname (no trailing comma) when the name is a single word.
    """
    if not isinstance(name, str):
        return ""
    parts = name.split()
    if not parts:
        return ""
    surname = parts[-1].title()
    given = ' '.join(parts[:-1]).title()
    return surname + ', ' + given if given else surname


def parse_group_ages(text):
    """
    Parses space-separated ages into a list of ints.

    Never raises. Returns [] when text is not a string. Any value that is not
    a whole non-negative number (letters, decimals, negatives) is skipped.
    """
    if not isinstance(text, str):
        return []
    ages = []
    for token in text.split():
        if token.isascii() and token.isdigit():
            try:
                ages.append(int(token))
            except ValueError:  # absurdly long digit strings
                continue
    return ages


def parse_visit_date(text):
    """
    Parses a YYYY-MM-DD date into a (year, month, day) tuple.

    Never raises. Returns None when text is not a string, does not have exactly
    two dashes, has non-numeric parts, or is not a real calendar date.
    """
    if not isinstance(text, str):
        return None
    parts = text.strip().split('-')
    if len(parts) != 3:
        return None
    if not all(p.isascii() and p.isdigit() for p in parts):
        return None
    try:
        year, month, day = int(parts[0]), int(parts[1]), int(parts[2])
        datetime.date(year, month, day)  # rejects month 13, Feb 30, year 0, ...
    except (ValueError, OverflowError):
        return None
    return year, month, day


def main():
    print("==== Running format_visitor_name ====")
    value = input("Enter a visitor's full name (e.g. asha rao): ")
    result = format_visitor_name(value)
    print("Name on ticket:", result if result else "Invalid name (no words entered)")

    print("\n==== Running parse_group_ages ====")
    value = input("Enter the ages in the group separated by spaces (e.g. 34 12 70): ")
    print("Group ages:", parse_group_ages(value))

    print("\n==== Running parse_visit_date ====")
    value = input("Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): ")
    result = parse_visit_date(value)
    if result is None:
        print("Invalid date (expected a real date as YYYY-MM-DD)")
    else:
        print("Visit date (year, month, day):", result)


if __name__ == "__main__":
    main()
```

Source code 2 – test_museum_admission_parser.py (Hypothesis fuzz tests)

```
from hypothesis import given, strategies as st
from museum_admission_parser import format_visitor_name, parse_group_ages, parse_visit_date


# Each test feeds arbitrary text (and None) to one function. A function passes
# only if it never raises, whatever the input. Any exception is caught with
# try/except and turned into a failure that names the crashing input.


@given(st.text() | st.none())
def test_format_visitor_name_no_crash(s):
    error = None
    try:
        result = format_visitor_name(s)
    except Exception as e:
        error = e
    assert error is None, (
        f"format_visitor_name({s!r}) crashed with {type(error).__name__}: {error}"
    )
    assert isinstance(result, str)


@given(st.text() | st.none())
def test_parse_group_ages_no_crash(s):
    error = None
    try:
        result = parse_group_ages(s)
    except Exception as e:
        error = e
    assert error is None, (
        f"parse_group_ages({s!r}) crashed with {type(error).__name__}: {error}"
    )
    assert isinstance(result, list)
    assert all(isinstance(age, int) and age >= 0 for age in result)


@given(st.text() | st.none())
def test_parse_visit_date_no_crash(s):
    error = None
    try:
        result = parse_visit_date(s)
    except Exception as e:
        error = e
    assert error is None, (
        f"parse_visit_date({s!r}) crashed with {type(error).__name__}: {error}"
    )
    assert result is None or (isinstance(result, tuple) and len(result) == 3)
```